hashlock.

Security Audit

YOM (NFT)

February 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

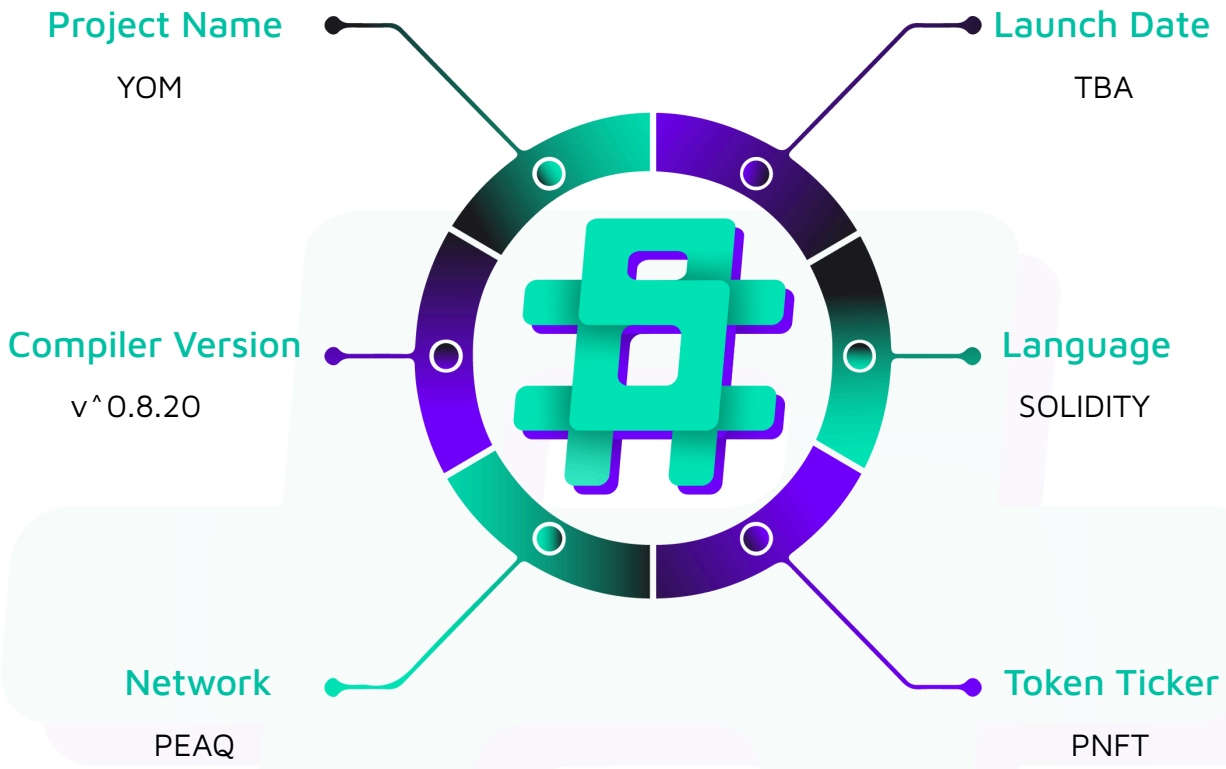
The YOM team partnered with Hashlock to conduct a security audit of their GenesisNodev2.sol smart contract. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts are secure.

Project Context

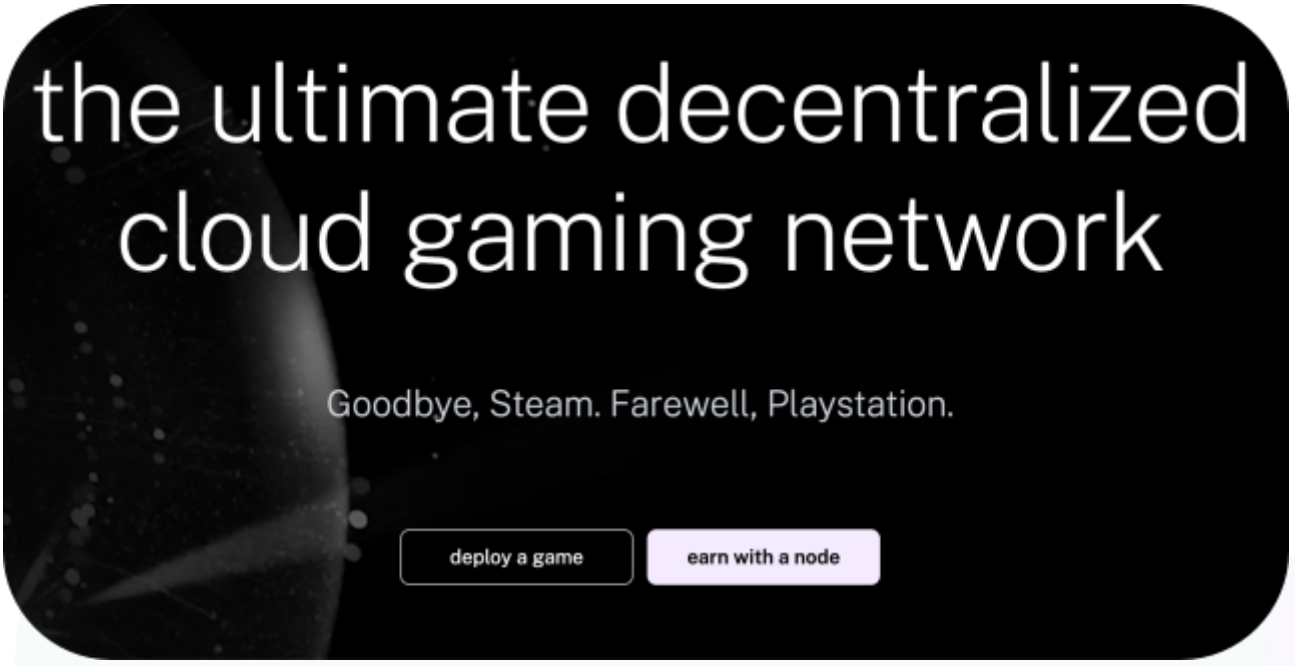
YOM is revolutionizing cloud gaming by pioneering the first decentralized cloud gaming infrastructure (DePIN). With YOM, games, immersive experiences, and virtual worlds can be streamed to any device, eliminating the need for expensive hardware like consoles (Xbox, PlayStation), powerful PCs, or dedicated apps (e.g., Steam, Apple Store).

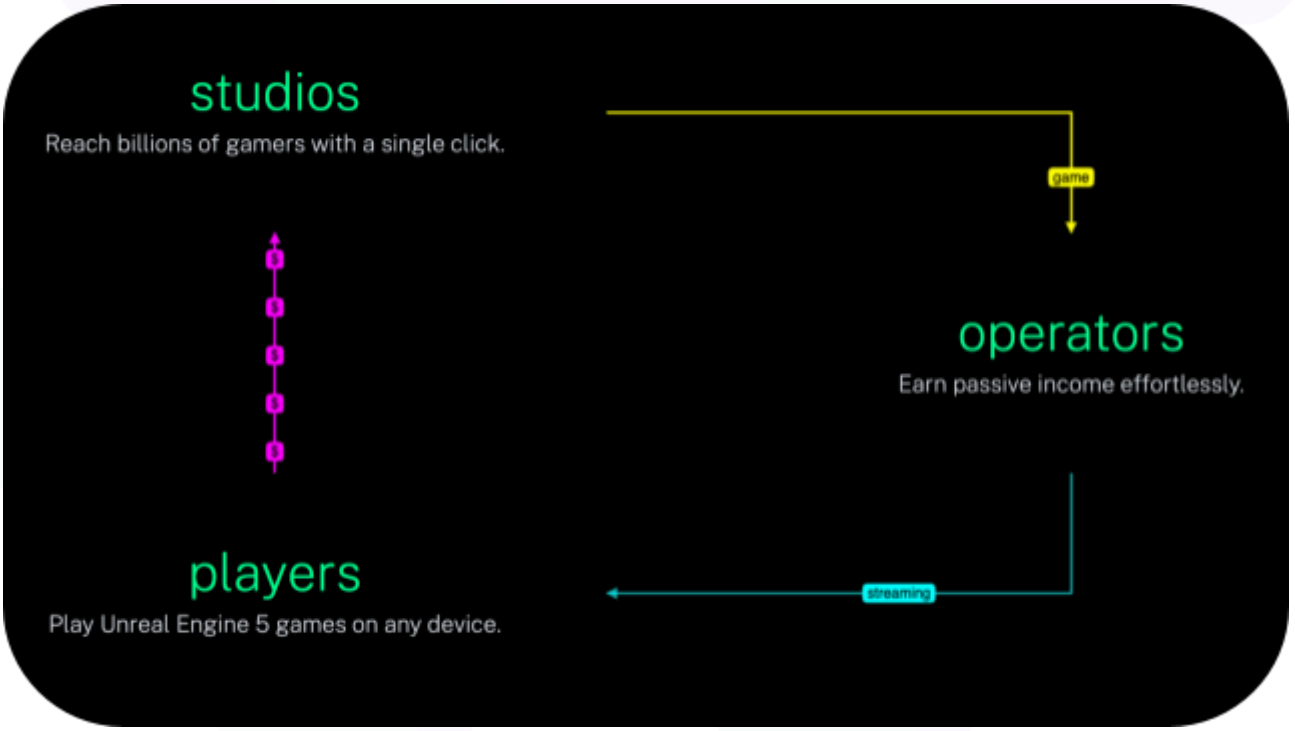
Project Name: YOM
Project Type: NFT
Compiler Version: ^0.8.20
Website: www.yom.net
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the YOM project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**YOM Protocol Smart Contracts**
Platform	**Peaq / Solidity**
Audit Date	**February, 2025**
Contract 1	GenesisNodev2.sol
Contract 1 MD5 Hash	9c82671a3d2180f3ecfa270bbc4a8c3d
Fix Review GitHub Commit Hash	ab9311945fe4e8d797f0e3070da0821210c8010e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified some vulnerabilities that need to be addressed prior to launch.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section, and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 Medium severity vulnerabilities

6 Low severity vulnerabilities

3 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

GenesisNodev2.sol
- Allows users to:
  - Mint new NFTs
  - Delegate/Undelegate keys
  - Access basic ERC721 functionality
- Allows admins to:
  - Add/Remove users from the allowlist
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the YOM project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the YOM project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] GenesisNodev2#safeMint - No restrictions on TokenURI set by users can lead to JSON injection and phishing attacks

Description

The `safeMint` allows an allowed minter to mint a new NFT with the URI they provide. There are no restrictions on what this URI can be. This can lead to problems such as URI pointing to malicious content, JSON injection, phishing attacks by impersonating different NFTs, and gas griefing attacks with a large URI.

Vulnerability Details

The `safeMint` allows an allowed minter to mint a new NFT with the URI they provide

```
function safeMint(address to, string memory uri) public {

        require(allowedMinters[msg.sender], "Not allowed to mint");

        uint256 tokenId = _nextTokenId++;

        _safeMint(to, tokenId);

        _setTokenURI(tokenId, uri);

        emit NFTTransferred(address(0), to, tokenId);

    }
```

As observed in this function, there are no restrictions on the user-inputted `uri` field. This allows any allowed minter to input malicious URIs, which can lead to problems such as:

- JSON injection
- Illegal content
- Phishing attacks via impersonating different NFTs

- Gas griefing attacks with a large URI

Impact

The impact of the issue depends on how the front-end systems sanitize the URIs. Without sanitization, malicious URIs can lead to cross-site scripting attacks, phishing attacks, and illegal content being present. On-chain, this can lead to gas griefing attacks and phishing attacks.

Recommendation

Do not allow arbitrary data to be inputted as the token URI. Make sure that the inputted URI is either:

- Generated by the contract
- It is a list of a mapping of valid URIs

Make sure that the inputted URI fits within a certain byte size and implement proper sanitizations on your front-end systems.

Alternatively, only allow the owner role to provide the token URI or make use of collections where the URI can be extracted.

Status

Resolved

[M-02] GenesisNodev2#setGreeting - No access control on the function can lead to harmful greeting messages and gas griefing attacks

Description

The `setGreeting` function allows any user to set the contract's greeting message. No access control in this function allows anyone to set harmful or illegal messages as the greeting message. Additionally, the greetings string can be a large string; other systems that make use of the greeting message can face serious gas costs if they make use of this message.

Vulnerability Details

The `setGreeting` function allows any user to set the contract's greeting message.

```
   function setGreeting(string memory _greeting) public {

        greeting = _greeting;

    }
```

As observed, there is no access control in this function. Any user can set the greeting message of the contract to harmful or illegal content. Additionally, there is no size limit on this string, which can lead to gas griefing attacks if external systems make use of this string.

Impact

A greeting message can be set to harmful or illegal content by any user.

No size limit on the greeting message allows large strings to be set as this message. This can lead to gas griefing attacks for external systems.

Recommendation

Implement access control and size limit in the function.

Status

Resolved

[M-03] GenesisNodev2#_burn - The burn functionality cannot be used

Description

The contract implements a `_burn` function; however, there are no user-facing functions that can make use of this `internal` function. Meaning that the burn mechanism will be unusable.

Vulnerability Details

The contract implements a `_burn` function

```
      function _burn(uint256 tokenId) internal virtual override(ERC721Upgradeable, ERC721URIStorageUpgradeable) {
        super._burn(tokenId);
    }
```

As observed, this function is `internal`, meaning that it needs another function to make use of it. However, there are no functions that make use of this burn functionality, making it unusable.

Impact

The burn functionality is unusable

Recommendation

Implement a new function with access control that calls the `_burn` function. However, consider the centralization risks this introduces.

Status

Resolved

[M-04] GenesisNodev2#_beforeTokenTransfer - NFT owners can delegate their tokens to prevent them from being burned

Description

The contract implements a `_burn` function; however, there are no user-facing functions that can make use of this `internal` function. When the burn functionality is properly implemented, a new issue arises where NFT owners can delegate their NFT to a user, making it impossible to burn this token.

Vulnerability Details

The `_beforeTokenTransfer` hook is applied before any token transfer, including a burn call

```
function _beforeTokenTransfer(
        address from,
        address to,
        uint256 firstTokenId,
        uint256 batchSize
    ) internal virtual override(ERC721Upgradeable) {
        require(!isKeyDelegated(firstTokenId), "Cannot transfer delegated token");
        super._beforeTokenTransfer(from, to, firstTokenId, batchSize);
        emit NFTTransferred(from, to, firstTokenId);
    }
```

As observed, if an NFT is delegated, no transfer of this token will be possible. This includes `_burn` calls that can be found in `ERC721Upgradeable`.

```
function _burn(uint256 tokenId) internal virtual {
        address owner = ERC721Upgradeable.ownerOf(tokenId);

        _beforeTokenTransfer(owner, address(0), tokenId, 1);
```

```
        // Update ownership in case tokenId was transferred by `_beforeTokenTransfer`
hook
        owner = ERC721Upgradeable.ownerOf(tokenId);

        // Clear approvals
        delete _tokenApprovals[tokenId];

        unchecked {
            // Cannot overflow, as that would require more tokens to be
burned/transferred
            // out than the owner initially received through minting and transferring in.
            _balances[owner] -= 1;
        }
        delete _owners[tokenId];

        emit Transfer(owner, address(0), tokenId);

        _afterTokenTransfer(owner, address(0), tokenId, 1);
    }
```

Impact

Tokens that are delegated will be impossible to burn; this can be used maliciously by NFT owners.

Recommendation

Implement checks in the `_beforeTokenTransfer` hook to bypass delegate checks if the tokens receiver is the 0 address.

Status

Resolved

Low

[L-01] GenesisNodev2#safeMint - Function emits the wrong event

Description

The `safeMint` function emits the `NFTTransferred` event. However, as this function mints the NFT, it should emit an event such as `NFTMinted`.

Recommendation

Implement a new `NFTMinted` event that will be emitted in this function.

Status

Resolved

[L-02] GenesisNodev2 - Use `Ownable2Step` instead of `Ownable`

Description

`Ownable2Step` prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g., due to a typo in the address) by requiring that the recipient of the owner permissions actively accept via a contract call of their own.

Recommendation

Consider using `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

[L-03] GenesisNodev2 - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Status

Resolved

[L-04] GenesisNodev2#addToAllowlist, addManyToAllowlist, removeFromAllowlist - Prevent setting a state variable with the same value

Description

Not only is it wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario.

Recommendation

Implement checks in the functions to prevent this scenario. An example is shown below.

```
   function addToAllowlist(address minter) public onlyOwner {
        require(allowedMinters[minter] == false, "Already a minter");
        allowedMinters[minter] = true;
        emit MinterAdded(minter);
    }
```

Status

Resolved

[L-05] GenesisNodev2 - Upgradeable contract is missing `__gap[..]` storage variable

Description

Storage gaps are needed to not breaking storage layouts when adding new variables to base contracts. Listed contracts may not be inherited, but it is good practice to add them now rather than forgetting later.

Recommendation

When designing upgradeable contracts, it's important to include `__gap[..]` storage variables as storage gaps to prevent issues with storage layout changes when adding new variables to base contracts in the future. While the listed contracts may not be directly inherited, it is a good practice to add `__gap[..]` storage variables now to ensure a robust and future-proof upgradeable contract design. This proactive approach helps avoid potential complications and storage layout conflicts in the later stages of contract development.

Status

Resolved

[L-06] GenesisNodev2 - Centralization risk for privileged functions

Description

Privileged functions need the privileged role to be trusted not to perform malicious updates. This may also result in a single point of failure. Functions with the centralization risk are shown below.

```
function addToAllowlist()

function addManyToAllowlist()

function removeFromAllowlist()

function _authorizeUpgrade()
```

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders.

Status

Acknowledged

Gas

[G-01] GenesisNodev2#URIUpdated - The event is unused

Description

The `URIUpdated` event is not used by the contract. It increases the deployment gas costs without a reason.

Recommendation

Remove the event.

Status

Resolved

[G-02] GenesisNodev2#addManyToAllowlist - Cache array length before looping

Description

Before iterating through an array with a for loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

Recommendation

Cache the array length by assigning it to a variable in memory before looping through the array.

Status

Resolved

[G-03] GenesisNodev2#addManyToAllowlist - Use unchecked for loop increments when overflow is impossible

Description

Solidity version 0.8.22 introduces an overflow check optimization that automatically generates an unchecked arithmetic increment of the counter of for loops. Since the contracts use 0.8.20, this optimization does not exist and causes a waste of gas.

Recommendation

Move the for loop increments into the loop body and make the increments unchecked. Alternatively, use Solidity version 0.8.22 for this optimization.

Status

Resolved

Centralisation

The YOM project values security and efficiency over decentralisation.

YOM emphasizes a security-focused approach while ensuring operational functionality. By centralizing critical functions, the project enhances reliability and rapid decision-making while maintaining accountability, ensuring the system remains robust and efficient.

Conclusion

After Hashlock's analysis, the YOM project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au